Technical Reference

Русский · User Guide · OKTMO

Contents

- Address Parsing
- Slots
- Matching Passes
- Settings
- Sources
- Run Artifacts
- Result Formatting
- Console Command
- Project Layout

How address parsing works, the slots, the matching passes, the configuration files, and the run artifacts.

Address Parsing

A raw address string is broken into components. The order matters: first the trailing building number is cut off, then the type and name of the street or territory are determined, then the settlement and the administrative unit above it.

What Is Recognised

Street types. улица, переулок, проспект, бульвар, тракт, шоссе, тупик, проезд, аллея, линия, набережная, площадь, квартал, микрорайон, километр, автодорога — with all the usual abbreviations (`ул.`, `пер.`, `пр-т`, `б-р`, `наб.`, `мкр.`).

Territory types. тер., территория, СНТ, ДНТ, ТСН.

Settlement types. город, посёлок, деревня, село, слобода, станица, станция, хутор, улус, местечко, кишлак, аул, аал, арбан, починок, выселок, заимка, кордон, маяк, погост, слободка, усадьба, лесоучасток, метеостанция, разъезд. Railway forms are parsed separately: `п. ж/д ст.`, `ж/д остановочный пункт`, `ж/д блокпост`, `ж/д будка`, `ж/д ветка`, `ж/д казарма`, `ж/д платформа`, `ж/д площадка`, `ж/д путевой пост`, `рзд.`. Compact forms `гп`, `рп`, `кп`, `дп`, `пгт`, `нп` are understood with and without dots.

Building number. The trailing part is split into a base and modifiers: `10`, `10а`, `10/2`, `10к2`, `10 корп. 2`, `10 стр. 1`, `10 литера А`. The base is the number; the modifiers are everything else, brought to one form.

The Key Rule About Building Numbers

Building modifiers can never be collapsed into a bare number. `10к2` does not take `10`, and `10` does not take `10к2` — neither on a strict pass nor on a relaxed one. The looseness of a pass concerns the spelling of the street, not blocks and letters.

Slots

A slot is one named component of an address. The address line is assembled from the enabled slots; in decomposed form each slot becomes a column.

Address Line Slots

Slot	Column	Short	What it is
`postal_index`	`Postal_Index`	`Index`	Postal index
`federal_district`	`Federal_District`	`Fed_District`	Federal district
`parent_subject`	`Parent_Subject`	`Parent`	Parent subject
`subject`	`Subject`	`Subject`	Federal subject
`autonomous_okrug`	`Autonomous_Okrug`	`AO`	Autonomous okrug
`municipality`	`Municipality`	`Municipality`	Municipality
`locality`	`Locality`	`Locality`	Settlement
`territory`	`Territory`	`Territory`	Territory, dacha association, and the like

Slot	Column	Short	What it is
`microdistrict`	`Microdistrict`	`Mkr_Qtr`	Microdistrict, quarter
`street`	`Street`	`Street`	Street
`house`	`House`	`House`	The building as a whole
`premise`	`Premise`	`Premise`	Premise, flat

Technical Slots

Slot	Column	Short	What it is
`oktmo_code`	`OKTMO_Code`	`OKTMO`	OKTMO code
`oktmo_name`	`OKTMO_Name`	`OKTMO_Name`	Official OKTMO name
`house_base`	`House_Base`	`House_No`	Building number without modifiers
`house_mods`	`House_Mods`	`House_Mods`	Modifiers in normalized form
`house_modifiers`	`House_Modifiers`	`House_Parts`	Modifiers as a list
`street_numbers`	`Street_Numbers`	`Street_Nums`	Numbers inside the street name
`territory_key`	`Territory_Key`	`Territory_Key`	Territory key
`match_key`	`Match_Key`	`Match_Key`	Matching key

Defaults

The default assembly order of the address line is the order of the address line slots in the table above, top to bottom.

Every slot is enabled by default except `microdistrict`: in Russian addresses a microdistrict or quarter more often duplicates the street than complements it. The microdistrict column still remains — only the value is excluded from the line.

Slot column headers are written in full or short form, chosen on the reference processing screen.

Matching Passes

The matching engine is `Ultimate_GT_Aligner`. For each address column a candidate pool is built, and the reference rows take their pairs out of it.

A matched pair is removed from the pool immediately. One candidate cannot be handed out twice, and a pass that ran earlier is never overridden by a later one.

The passes run in strictly increasing looseness.

Pass	Name	Match condition
0	Source row	Collection-before-match mode only: the candidate comes from the same source row and its components agree
1	Exact string	The normalized address texts are equal
2	Component hash	Territory, street words, street numbers, building number, and modifiers all match
2R	Relaxed hash	The same without modifiers in the key — but the building check still requires matching modifiers
2C	Core hash	Collection mode only: street and building without territory. Requires a strict building check and compatible territory codes
3	Fuzzy, inside a territory	Street name similarity within one territory cluster, cutoff 85
4	Fuzzy, across territories	The same across the whole pool, cutoff 85, with weights
5	Relaxed	Cutoff 80; building modifiers are still protected

Precise passes are 0, 1, 2, 2C, 3, and 4. **Relaxed** are 2R and 5. The log summary separates them into their own lines.

Weights on the Fuzzy Passes

At equal street name similarity a candidate receives bonuses:

- matching building number — plus 50;
- matching territory — plus 30;
- matching building modifiers (pass 5) — plus 20;

- matching OKTMO territorial context — a bonus from the search scope profile.

A pair with incompatible territory codes is discarded before weights are counted.

Rows Without a Street

When a reference row has a building number but no street, comparing names approximately is meaningless. Such rows look for a candidate with the same building number, modifiers, and street numbers, and pick the best one by territorial weights.

Settings

`config/project.yaml`

```
address_aligner:
  city: "тюмень"              # city removed from keys during normalization
  ground_truth_column: "B"    # reference column; "" = auto-detect
  target_columns: "C,D"       # address columns; "" = auto
  companion_mode: "auto"      # auto | none | left | right | between
  max_auto_align_columns: 5   # how many columns to take on the right in auto mode

safety:
  cleanup_managed_workspace_only: true   # clean only project folders
  never_delete_original_inputs: true     # never delete original sources
```

Values from the window take priority over the file: an empty field in the window means "take from settings or detect automatically", a filled one overrides.

`config/window_settings.json`

The window's settings: its size, the language (`ru` | `en`), the theme (`dark`, `codedark`, `vault`, `light`), the font size and the width of the right column. The window writes the file itself; delete it to return to the defaults (Russian, the Code Dark theme). At start the source and the target are always the project's `input` and `output`: the earlier choice is not remembered, the history of paths is kept apart.

Other Configuration Files

File	Contents
`config/tool_manifest.yaml`	Every operation, field, hint, and service function
`config/window_layout.yaml`	The layout of the window: the sections, the operations in them, the buttons of the working area and of the result panel, the icons
`config/version.json`	Version and the language of the root README
`config/path_history.json`	History and pins of source and target paths
`config/oktmo_current_keys.txt`	OKTMO search keys
`config/oktmo_*_pins.json`	Pinned regions and municipalities

`tool_manifest.yaml` is what the window is built from. Each operation is described by a title, a description, a set of fields, and a reference to a service-layer function. Both languages live in the same file as `label` / `label_ru` pairs. The window knows no operation by name: what to draw is said by the keys of the manifest and by the layout `window_layout.yaml`. Every operation of the manifest must stand in the layout — the tests and the self-check of the window verify it.

Sources

Supported: `.xlsx`, `.docx`, `.odt`, `.txt`, `.csv`, `.md`, `.markdown`, and `.pdf` with a text layer. A PDF without a text layer is not recognised — that is not this program's job.

Legacy `.doc` and `.xls` are handled only after conversion by a separate command.

Office temporary files (`~$*`) are skipped and removed during input validation.

Two-Level Headers

A row that carries no data values and sits under a horizontally merged parent header counts as a sub-header row. Its values are joined to the parent headers, and the row itself does not reach the data.

A column headed "Проектная мощность, мест" over "по корпусам" becomes "Проектная мощность, мест по корпусам" instead of an unnamed `column_5`.

Run Artifacts

Results

Prefix	What it is
`AddressCollection_*.xlsx`	Collected address workbook
`AddressReference_*.xlsx`	Reference directory

Names carry a `YYYYMMDD_HHMMSS` timestamp.

Reports in `report/`

File	Run
`address_collection_summary.json`	Address collection
`address_collection_benchmark.json`	Collection scored against a control column
`address_alignment_summary.json`	Address alignment
`reference_normalization_summary.json`	Reference normalization
`reference_processing_summary.json`	Reference processing
`table_comparison_summary.json`	Comparison of two tables
`safe_table_join_summary.json`	Moving columns by address
`legacy_office_conversion_report.json`	DOC/XLS conversion
`rosstat_oktmo_update_summary.json`	OKTMO registry update

Some operations additionally write `<result name>.summary.json` next to the workbook.

Audit Sheets

Moving columns by address adds an `AUDIT_ADDRESS_JOIN` sheet to the result. Address collection writes a rejected sheet when that option is on.

Result Formatting

Result workbooks go through post-processing: column widths are fitted to the content, long text wraps, row heights grow, and status columns get colour fill.

This is not cosmetics: without it every export has to be tidied by hand, and a row with a long address cannot be read in a narrow column.

Console Command

Whatever a button of the window does, the console command `address_cli.cmd` does too: the window is its "face" and has no way of its own to run an operation. An operation runs with the same field defaults and the same rules, so a run from the command line gives the same workbook as a run from the window.

Command	What it does
`describe`	The manifest as JSON: operations, fields, defaults, both languages, the window layout
`run --operation ID`	Run an operation; keys `--source`, `--target`, `--params file.json`, `--set name=value`, `--result`, `--cancel-file`, `--session-dir`
`options --operation ID --field ID`	The options of a field the engine builds (sheets and headers of workbooks, regions and municipalities)
`oktmo --action NAME`	The actions of the OKTMO panel: registry, keys, pins, settlement search
`serve`	A process that stays up: the same `options` and `oktmo` as lines of JSON through stdin/stdout, so the OKTMO registry is not read again for every question
`stop-office --session-dir`	Close the Excel and Word that a stopped operation started (only those recorded in the session folder — the person's own Excel and Word are never touched)
`version`	The number from `config/version.json`

An ordinary line of output goes to the journal. A line that begins with `::address::` is a service one and carries JSON: where the operation's journal is written, the share of the work done, the live line of a long stage, the outcome. The outcome of an operation is the `--result` file; stopping is the `--`

`cancel-file`: the operation sees it and ends at the nearest safe point. Exit codes: 0 — done, 1 — the operation failed, 2 — a wrong call, 130 — stopped.

The rules that assemble the parameters of a run (the defaults, the conditions for showing a field, the order of the slots, the reset of the normalization fields in the "fast" mode, values outside a list) live in one place — `system_core/core/operation_model.py` — and the tests hold them.

Project Layout

Layer	Where	Responsibility
Window	`Source/` (sources), `App/` (built)	Avalonia 12 / .NET 10: sections, forms, journal, result, working folders
Console command	`system_core/cli.py`, `system_core/core/operation_model.py`	One way to start an operation for the window and the command line
Services	`system_core/services/`	Binding the window to the engines, reports, parameters
Address engine	`system_core/address_engine/`	Parsing, slots, OKTMO, reference, collection
Matching	`system_core/Ultimate_GT_Aligner.py`	Matching passes and linked column transfer
Join by address	`system_core/safe_table_join.py`	Table comparison and column transfer through Excel
Core	`system_core/core/`	Jobs, paths, encodings, manifest

A detailed file map is in tools/PROJECT_FILES_GUIDE_EN.md.